

ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

LAB ASSIGNMENT – 1

NAME : SUFIYA AFREEN

ENROLLMENT NUMBER : 2403A51067

BRANCH AND BATCH : CSE – 03

YEAR/SEM – 3/1

DATE : 5-8-26

Lab/Exp 1: PyTorch Fundamentals: Tensors and Autograd

Kaggle Dataset Link: <https://www.kaggle.com/datasets/larsen0966/student-performance-dataset>

Tasks:

1. Load and preprocess the Student Performance dataset/example.
2. Build the required PyTorch model/program.
3. Train/execute the model.
4. Evaluate using suitable metrics.
5. Visualize results using plots where applicable.
6. Write a brief conclusion comparing observations.

CODE :

```
import os
import math
import zipfile
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# 1) Set seed for reproducibility
seed = 42
np.random.seed(seed)
torch.manual_seed(seed)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# 2) Load dataset

csv_path = "/content/archive (1).zip"

df = pd.read_csv(csv_path)
print("Shape:", df.shape)
print(df.head())

# 3) Basic preprocessing
target_col = "math score"

# Standardize column names for convenience
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]

# The actual target column from the dataset after standardization should be 'g3'
# if it represents the final math score. The original dataset columns were G1, G2, G3.
target_col = "g3"

# Handle categorical columns with label encoding
categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()
encoders = {}
```

```

for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    encoders[col] = le

# Separate features and target
X = df.drop(columns=[target_col]).copy()
y = df[target_col].copy()

# Standardize numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y.values, test_size=0.2, random_state=seed
)

# Further split train into train/val
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=seed
)

print("Train:", X_train.shape, "Val:", X_val.shape, "Test:", X_test.shape)

# 4) Torch Dataset and DataLoader
# -----
class StudentDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32).view(-1, 1)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

batch_size = 32

train_ds = StudentDataset(X_train, y_train)
val_ds = StudentDataset(X_val, y_val)
test_ds = StudentDataset(X_test, y_test)

train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_ds, batch_size=batch_size, shuffle=False)

```

```
3)  # 5) Build PyTorch model
# -----
class StudentModel(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )

    def forward(self, x):
        return self.net(x)

input_dim = X_train.shape[1]
model = StudentModel(input_dim).to(device)

# Loss and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

print(model)

# 6) Train function
# -----
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss = 0.0

    for X_batch, y_batch in loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()
        preds = model(X_batch)
        loss = criterion(preds, y_batch)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * X_batch.size(0)

    return total_loss / len(loader.dataset)
```

```
# 7) Evaluation function
# -----
def evaluate(model, loader, criterion):
    model.eval()
    total_loss = 0.0
    all_preds = []
    all_targets = []

    with torch.no_grad():
        for X_batch, y_batch in loader:
            X_batch = X_batch.to(device)
            y_batch = y_batch.to(device)

            preds = model(X_batch)
            loss = criterion(preds, y_batch)

            total_loss += loss.item() * X_batch.size(0)
            all_preds.extend(preds.cpu().numpy().flatten())
            all_targets.extend(y_batch.cpu().numpy().flatten())

    avg_loss = total_loss / len(loader.dataset)
    return avg_loss, np.array(all_targets), np.array(all_preds)

# 8) Train the model
# -----
epochs = 100
train_losses = []
val_losses = []

for epoch in range(1, epochs + 1):
    train_loss = train_one_epoch(model, train_loader, criterion, optimizer)
    val_loss, _, _ = evaluate(model, val_loader, criterion)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    if epoch % 10 == 0 or epoch == 1:
        print(f"Epoch {epoch:3d}/{epochs} | Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")
```

```
# 9) Test evaluation metrics
# -----
test_loss, y_true, y_pred = evaluate(model, test_loader, criterion)

mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = math.sqrt(mse)
r2 = r2_score(y_true, y_pred)

print("\nTest Results")
print("MSE :", round(mse, 4))
print("RMSE:", round(rmse, 4))
print("MAE :", round(mae, 4))
print("R2  :", round(r2, 4))

# 10) Visualizations
# -----
plt.figure(figsize=(8, 5))
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Val Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 7))
plt.scatter(y_true, y_pred, alpha=0.7)
min_val = min(y_true.min(), y_pred.min())
max_val = max(y_true.max(), y_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], "r--")
plt.xlabel("Actual Math Score")
plt.ylabel("Predicted Math Score")
plt.title("Actual vs Predicted")
plt.grid(True)
plt.show()

# Optional: residual plot
residuals = y_true - y_pred
plt.figure(figsize=(8, 5))
plt.scatter(y_pred, residuals, alpha=0.7)
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Predicted")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.grid(True)
plt.show()

# 11) Brief conclusion
# -----
print("\nConclusion:")
print(
    f"The PyTorch model learned patterns in the student performance data and produced a test MAE of {mae:.2f} "
    f"and R2 of {r2:.2f}. The loss curves show how autograd helped optimize the network during training. "
    "If performance is not strong enough, you can improve it by predicting multiple scores, tuning layers, "
    "or trying a classification target instead."
)
```

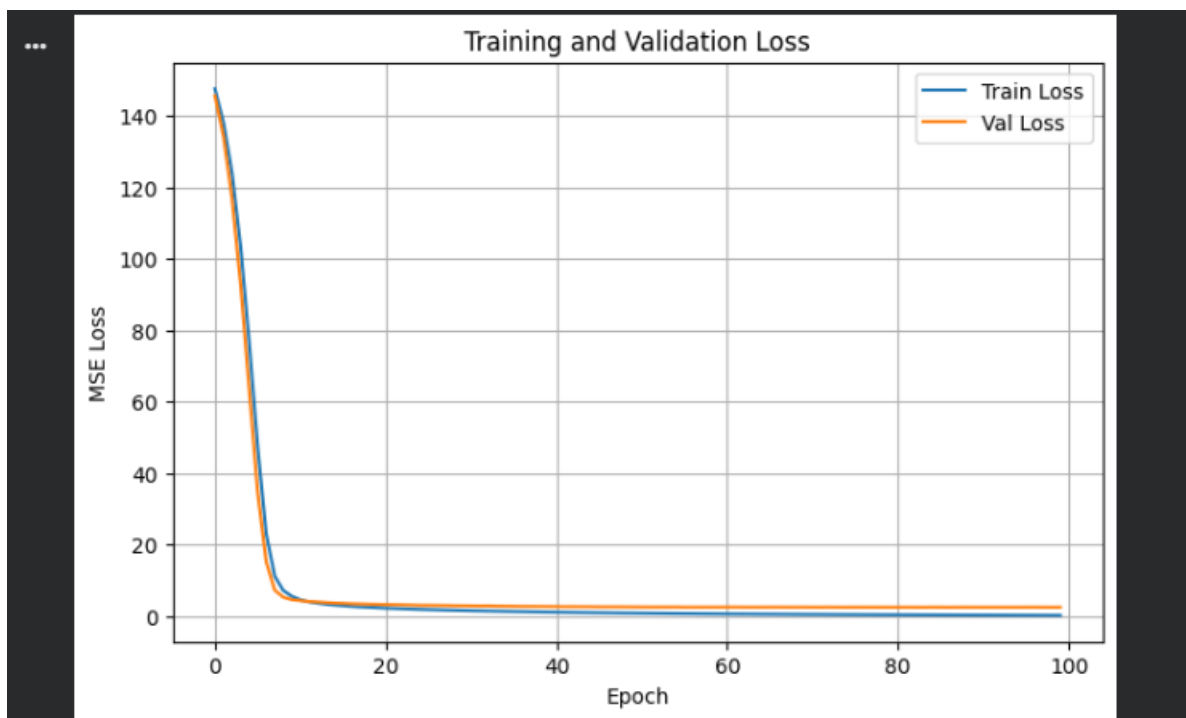
OUTPUT:

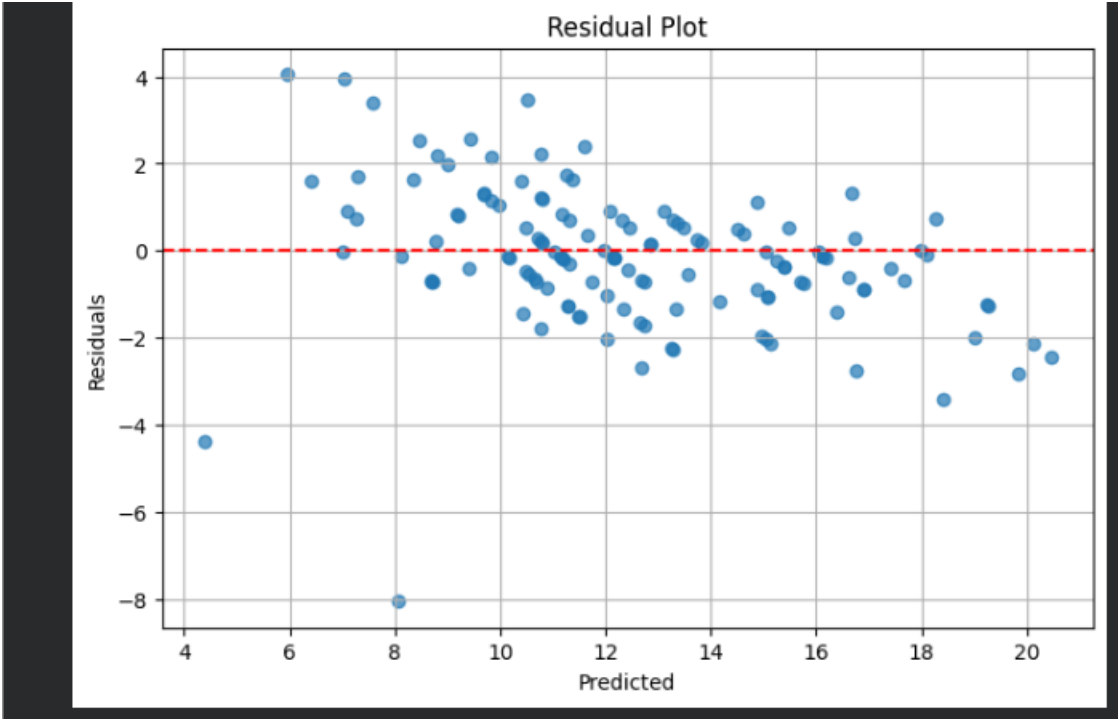
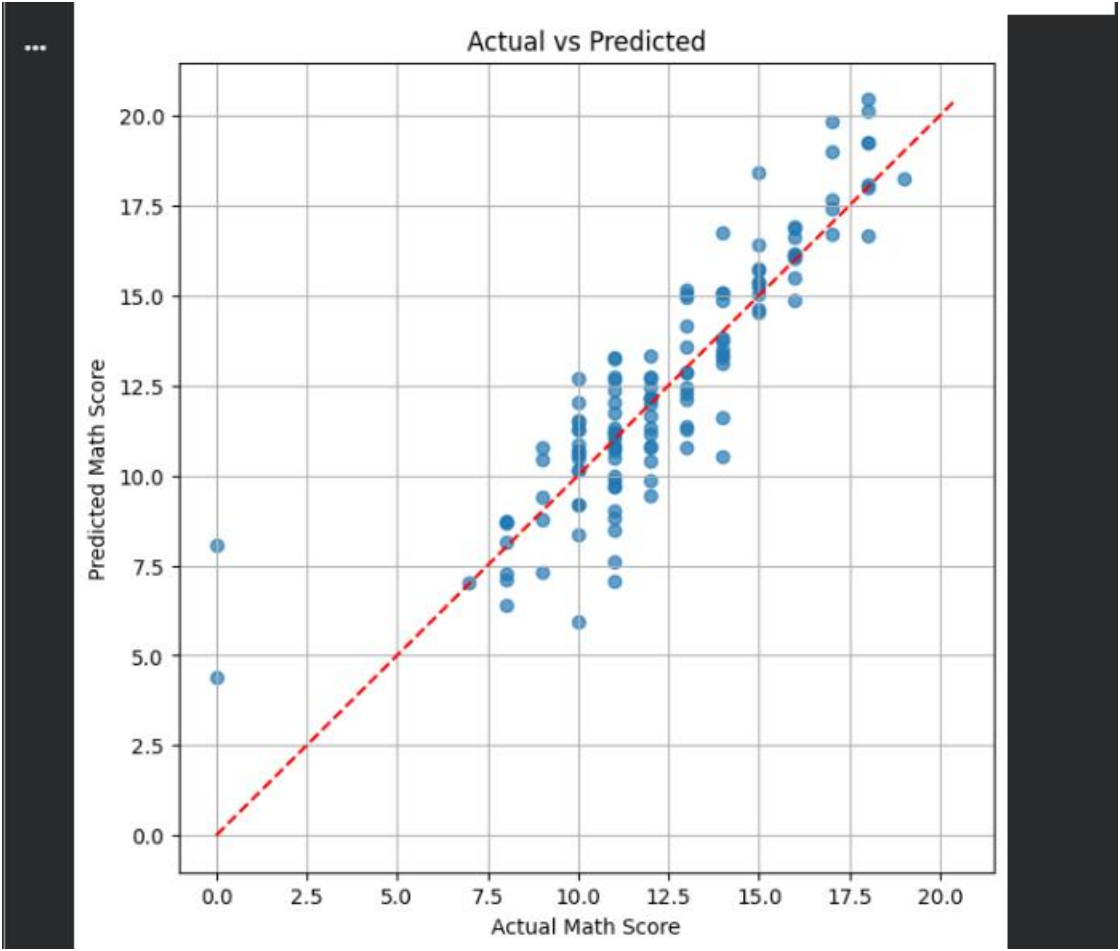
```
Device: cpu
Shape: (649, 33)
...
  school sex age address famsize Pstatus Medu Fedu Mjob Fjob ... \
0 GP F 18 U GT3 A 4 4 at_home teacher ...
1 GP F 17 U GT3 T 1 1 at_home other ...
2 GP F 15 U LE3 T 1 1 at_home other ...
3 GP F 15 U GT3 T 4 2 health services ...
4 GP F 16 U GT3 T 3 3 other other ...

famrel freetime goout Dalc Walc health absences G1 G2 G3
0 4 3 4 1 1 3 4 0 11 11
1 5 3 3 1 1 3 2 9 11 11
2 4 3 2 2 3 3 6 12 13 12
3 3 2 2 1 1 5 0 14 14 14
4 4 3 2 1 2 5 0 11 13 13

[5 rows x 33 columns]
Train: (415, 32) Val: (104, 32) Test: (130, 32)
StudentModel(
  (net): Sequential(
    (0): Linear(in_features=32, out_features=64, bias=True)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=1, bias=True)
  )
)
Epoch 1/100 | Train Loss: 147.5863 | Val Loss: 145.6809
Epoch 10/100 | Train Loss: 5.5345 | Val Loss: 4.5982
Epoch 20/100 | Train Loss: 2.3443 | Val Loss: 3.2779
Epoch 30/100 | Train Loss: 1.5707 | Val Loss: 2.8867
Epoch 40/100 | Train Loss: 1.1275 | Val Loss: 2.6625
Epoch 50/100 | Train Loss: 0.8459 | Val Loss: 2.5585
Epoch 60/100 | Train Loss: 0.6328 | Val Loss: 2.5083
Epoch 70/100 | Train Loss: 0.4825 | Val Loss: 2.5056
Epoch 80/100 | Train Loss: 0.3715 | Val Loss: 2.4844
Epoch 90/100 | Train Loss: 0.2833 | Val Loss: 2.4876
Epoch 100/100 | Train Loss: 0.2130 | Val Loss: 2.4860

Test Results
MSE : 2.5671
RMSE: 1.6022
MAE : 1.152
R2 : 0.7367
```





CONCLUSION :

This dataset shows that student performance can be modeled from simple demographic and preparation-related features, but the prediction quality depends on how strongly those features relate to the target score. In the code, the **PyTorch fundamentals** used were tensors, `autograd`, `nn.Module`, loss functions, optimizers, and the training loop, which together form the core workflow for neural network training in PyTorch. PyTorch's autograd engine automatically computes gradients for the computational graph, making backpropagation and model updates straightforward.